dove verrebbero definite le label applicative ? Queste da sole permetterebbero di stabilire la compatibilità e quindi implementare la politica proposta al punto 6) ?

Le label applicative verrebbero definite dal codice comune di `docker-dev` e inserite nella sezione `labels` del servizio `dev` nel Compose generato.

Da sole, però, **non bastano** a stabilire tutta la compatibilità e ad applicare la politica proposta. Servono a identificare il container e a registrare la configurazione con cui è stato creato; devono essere affiancate dallo stato Docker e da controlli sull'immagine e sulla configurazione effettiva.

1. Dove definirle

Separerei la definizione delle label dai valori che assumono:

Elemento	Dove risiede
Nomi delle label e loro significato	Modulo Perl comune di `docker-dev`
ID del progetto e lifecycle richiesto	Contesto risolto
Impronta della configurazione	Calcolata dal resolver
Label da applicare al container	`services.dev.labels` nel Compose generato

Per esempio:

```
services:
  dev:
    image: dev.python

    labels:
      dev.docker-dev.managed: "true"
      dev.docker-dev.project-id: "project-123"
      dev.docker-dev.role: "dev"
      dev.docker-dev.lifecycle: "persistent"
      dev.docker-dev.runtime-schema: "1"
      dev.docker-dev.runtime-hash: "sha256:..."
```

Il prefisso è illustrativo: va scelto una volta come namespace del tool, evitando quelli riservati da Docker. Le label sono coppie chiave/valore stringa e quelle dei container restano statiche per tutta la vita dell'oggetto. [1]

Non le metterei:

- nel Dockerfile, perché la stessa immagine serve più progetti;
- nel `runtime.yaml` di linguaggio, perché sono metadati dell'orchestratore;
- nel `runtime.yaml` del progetto come valori liberamente modificabili.

Il template Compose può ricevere una mapping `managed_labels` già preparata dal codice Perl. Le personalizzazioni dell'utente non devono poter sovrascrivere le label riservate al tool.

2. Che cosa permettono di verificare

Identità

Queste label permettono di verificare che il container dichiari di essere:

```
gestito da docker-dev
+
appartenente al progetto atteso
+
destinato al ruolo dev
```

Sono sufficienti per un controllo operativo di appartenenza nel nostro ambiente, ma non costituiscono una prova di sicurezza: un container creato manualmente può riportare le stesse label.

Lifecycle

La label:

```
dev.docker-dev.lifecycle: persistent
```

registra l'intenzione al momento della creazione. È utile per distinguere un container persistente da uno creato per una sessione effimera.

Configurazione di creazione

La label `runtime-hash` registra l'impronta del modello runtime risolto utilizzato per creare il container.

Poiché le label non si aggiornano quando cambia il file runtime, possiamo confrontare:

```
hash richiesto oggi
        contro
hash registrato alla creazione
```

Questa staticità è utile: la label rappresenta la configurazione originaria, non quella attualmente presente nei file del progetto.[1]

3. Che cosa le label non descrivono

Non ci dicono direttamente:

- se il container è fermo, attivo o in riavvio;
- quale ID immagine sta effettivamente usando;
- quali mount sono realmente applicati;
- se i servizi dipendenti sono pronti;
- se l'utente ha modificato il filesystem interno;
- se l'ambiente applicativo è ancora funzionante.

Per questi controlli serve `docker container inspect`, che restituisce informazioni dettagliate sul container, anche quando è fermo.[2] [3]

Quindi il controllo completo è:

```
label di identità
        +
configurazione richiesta risolta
        +
configurazione e stato effettivi del container
```

4. Come rendere utile il `runtime-hash`

L'hash deve essere calcolato su un modello normalizzato, non sul testo YAML.

Per esempio, cambiare l'ordine delle chiavi o aggiungere un commento non deve rendere il container incompatibile.

Il modello da confrontare dovrebbe includere:

- ID dell'immagine risolta, non soltanto il tag;
- mount risolti;
- ambiente previsto;
- porte;
- hostname;
- utente e working directory;
- entrypoint e comando iniziale;
- risorse e opzioni di sicurezza supportate;
- configurazione di rete rilevante;
- modalità TTY/stdin quando fa parte della configurazione di creazione.

Va esclusa la label `runtime-hash` stessa, altrimenti il calcolo diventerebbe circolare.

Va inoltre registrata la versione dell'algoritmo o dello schema del fingerprint: un aggiornamento del resolver non deve produrre incompatibilità inspiegabili.

Perché includere l'ID immagine

Se il tag `dev.python` viene ricostruito, il nome resta uguale ma l'immagine può cambiare. La verifica deve confrontare l'immagine del container con quella attualmente risolta dal tool.

Docker permette di ispezionare separatamente container e immagini; il confronto deve usare i loro identificativi effettivi, non dedurre l'equivalenza dal nome.[2]

5. Controllo consigliato nella prima versione

Non implementerei subito un confronto esaustivo di ogni campo Docker. Adotterei tre livelli:

A. Identità obbligatoria

Verificare:

```
managed
project-id
role
```

Se mancano o non corrispondono, nessun riutilizzo automatico.

Per i container già esistenti senza label, prevedere eventualmente un'adozione esplicita o una ricreazione: non considerarli automaticamente compatibili.

B. Compatibilità dichiarativa

Verificare:

```
versione fingerprint supportata
runtime-hash uguale
lifecycle coerente
ID immagine uguale
```

Questo copre la maggior parte dei cambiamenti introdotti attraverso `docker-dev`.

C. Verifiche effettive essenziali

Controllare mediante inspect almeno:

- stato;
- mount standard e loro destinazioni;
- immagine;
- assenza di condizioni che impediscono il normale avvio.

Il controllo non deve certificare che il software applicativo sia “sano”: stabilisce se il container può essere riutilizzato secondo la configurazione richiesta.

6. Applicazione alla politica proposta

Situazione	Informazioni necessarie
Container assente	Ricerca per nome/ID
Container di un altro progetto	Label di identità
Container fermo e compatibile	Label, fingerprint, immagine e stato
Container attivo e compatibile	Stessi controlli, più scelta esplicita tra attach ed exec
Container incompatibile	Differenza di fingerprint, immagine o configurazione verificata
Cambio da persistente a effimero	Lifecycle registrato contro lifecycle richiesto

Le label permettono quindi di implementare una parte importante della politica, ma lo stato e la configurazione effettiva completano la decisione.

7. Non perdiamo il motivo del container persistente

Un container persistente può contenere modifiche manuali al filesystem interno. Un hash uguale non prova che tali modifiche non esistano — e non dovrebbe necessariamente farlo.

Dobbiamo distinguere:

- compatibilità di creazione: il container è stato creato con il runtime richiesto;
- integrità dell’ambiente interno: il contenuto del container non è cambiato;
- salute applicativa: gli strumenti e l’applicazione funzionano.

Per il riutilizzo proposto ci interessa soprattutto la prima. Se volessimo garantire anche l’integrità, torneremmo verso un modello di ricreazione sistematica, riducendo il beneficio della persistenza.

La soluzione proporzionata è dunque: label gestite automaticamente nel Compose generato, fingerprint del runtime e controlli inspect essenziali. Questo consente un riutilizzo ragionato senza trasformare `docker-dev` in un sistema complesso di verifica dell’intero container.

⁂

1. https://docs.docker.com/engine/manage-resources/labels/
2. https://docs.docker.com/reference/cli/docker/inspect/
3. https://docs.docker.com/reference/cli/docker/container/inspect/
4. https://docs.docker.com/reference/cli/docker/compose/
5. https://docs.docker.com/reference/compose-file/
6. https://docs.docker.com/reference/cli/docker/compose/ps/
7. https://docs.docker.com/reference/cli/docker/compose/up/

8. https://docs.docker.com/compose/releases/release-notes/
9. https://docs.docker.com/engine/storage/bind-mounts/
10. https://docs.docker.com/reference/cli/docker/container/ls/
11. https://docs.docker.com/desktop/use-desktop/container/
12. https://docs.docker.com/compose/intro/history/
13. https://docs.docker.com/compose/how-tos/startup-order/
14. https://docs.docker.com/reference/compose-file/services/
15. https://docs.docker.com/compose/intro/compose-application-model/